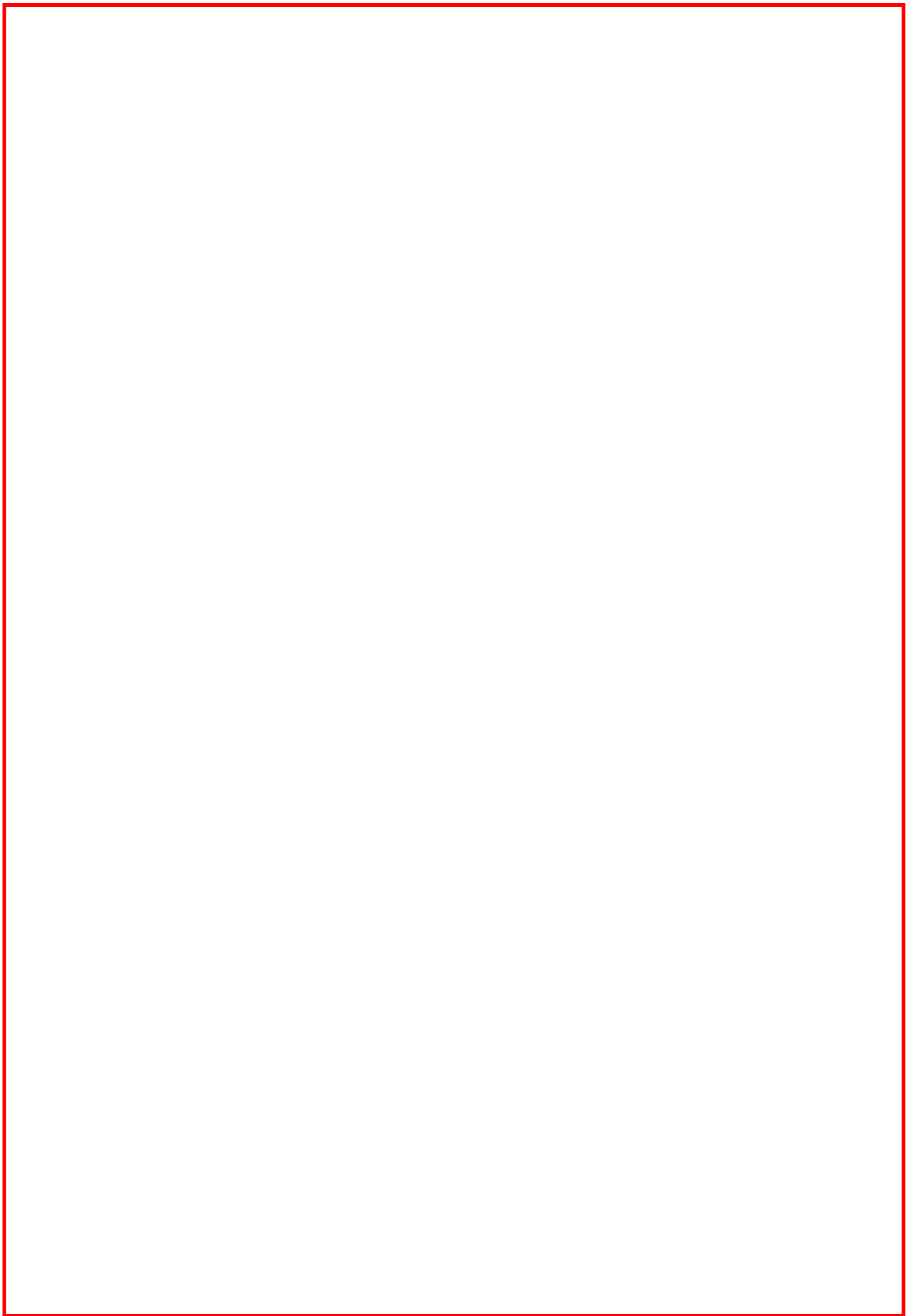




HOSPITAL MANAGEMENT SYSTEM
SOFTWARE ENGINEERING PROJECT REPORT

B.Sc(H) Computer Science



Submitted by:-

Aashish	Ankur Tyagi	Deepanshu Khatri	Himanshu Baldodia	Keshav Singh
16035500000	16035500000	16035500000	16035500000	16035500000

Department of Computer Science

Keshav Mahavidyalaya

University of Delhi

ACKNOWLEDGEMENT

Apart from the efforts of team, the success of any project depends largely on the encouragement and guidelines of many others. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project.

The completion of any inter-disciplinary project depends upon cooperation, co-ordination and combined efforts of several sources of knowledge.

We are eternally grateful to our **Dr. Sumit Agarwal** for his even willingness to give us valuable advice and direction under whom we executed this project. His constant guidance and willingness to share his vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks.

We are highly thankful to our project internal guide **Mrs. Disha**, and **Mr. Anand** whose invaluable guidance helped us understands the project better.

Although there may be many who remain unacknowledged in this humble note of gratitude, there are none who remain unappreciated.

Aashish (16035500000)

Ankur Tyagi (16035500000)

Deepanshu Khatri (16035500000)

Himanshu Baldodia (16035500000)

Keshav Singh (16035500000)

CERTIFICATE

This is to certify that Software Engineering project report entitled "Hospital Management System" is the work carried out by **Aashish, Ankur Tyagi, Deepanshu Khatri, Himanshu Kumar** and **Keshav Singh**, students of BSc(H) Computer Science IV Sem, Keshav Mahavidyalaya, University of Delhi under the supervision of **Dr.Sumit Agarwal, Mr.Anand, Mrs.Disha**.

This report has not been submitted to any other organisation/institution for the award any other degree/diploma.

Supervisor

Dr.Sumit Agarwal

Teacher in charge

Ms.Vinita Jindal

Principal

Dr.Madhu Pruthi

ABSTRACT

Our project Hospital Management system includes registration of patients, storing their details into the system, and also booking their appointments with doctors . Our software has the facility to give a unique id for every patient and stores the details of every patient and the staff automatically. User can search availability of a doctor and the details of a patient using the id. The Hospital Management System can be entered using a username and password. It is accessible either by an administrator or receptionist. Only they can add data into the database. The data can be retrieved easily. The interface is very user-friendly. The data are well protected for personal use and makes the data processing very fast.

It is having mainly two modules. One is at Administration Level and other one is of user I.e. of patients and doctors. The Application maintains authentication in order to access the application. Administrator task includes managing doctor's information, patient's information. To achieve this aim a database was designed one for the patient and other for the doctors which the admin can access. The complaints which are given by user will be referred by authorities.

The user modules include checking appointments, prescription. user can also register complaint against management or ask any query.

LIST OF IMAGES USED IN THE PROJECT

NAME	PAGE NO
1. DATA FLOW DIAGRAM	05
2. ARCHITECTURAL DESIGN	23
3. LOGIN PAGE	25
4. REGISTRATION	26
5. PATIENT	27
6. DOCTOR VIEW APPOINTMENT	30
7. DOCTOR VIEW PATIENT	31
8. DOCTOR ADD DESCRIPTION	32

LIST OF TABLES USED IN THE PROJECT

NAME	PAGE NO
1.DATA DICTIONARY	6
1.SIZE ESTIMATION	13
2.GANTT CHART	21
3.RISK ANALYSIS	24

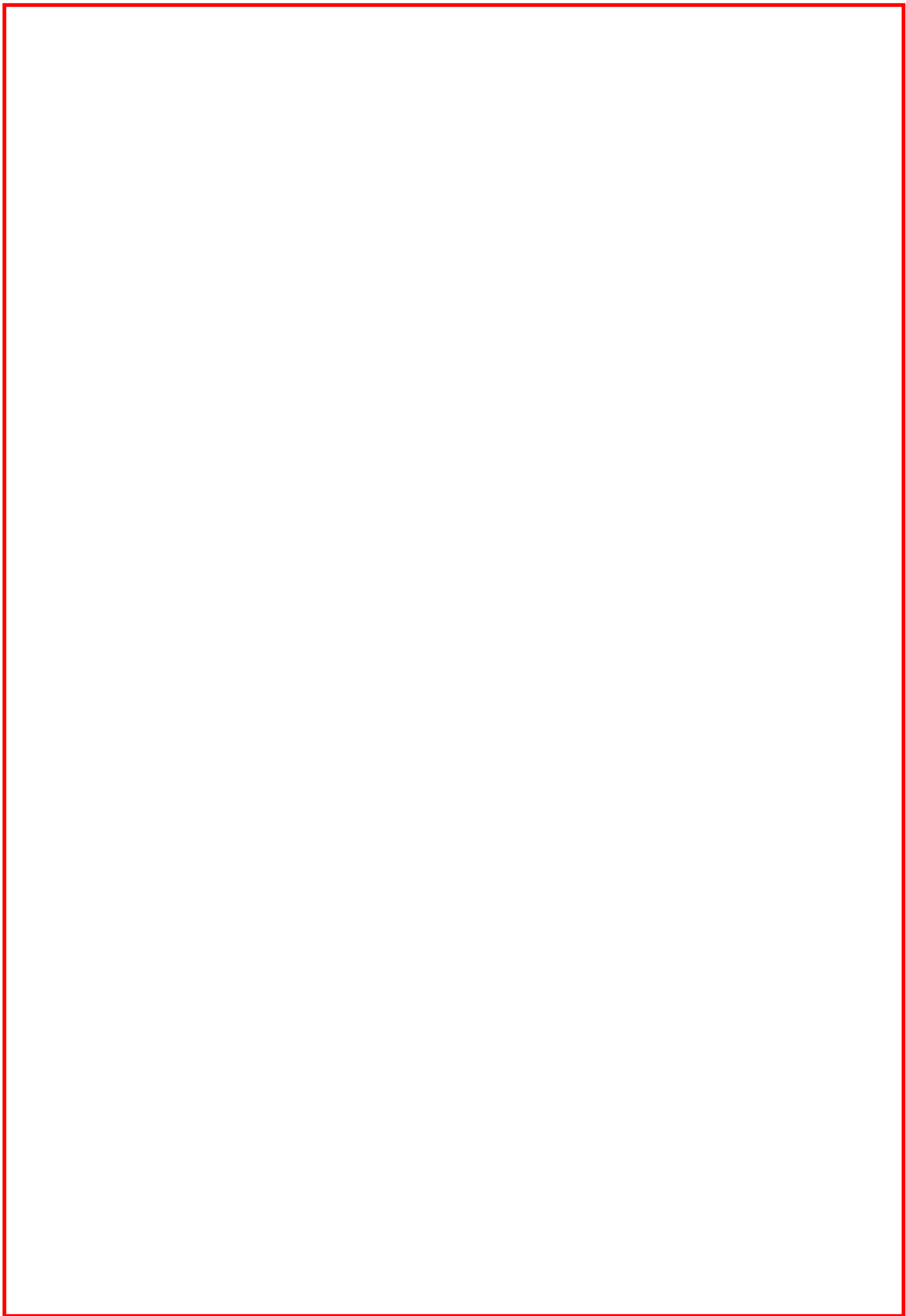


Table of Content

1. CHAPTER 1 SRS (Software Requirement Specification)

1.1 Introduction.....	1
1.1.1 Purpose.....	1
1.1.2 Document Conventions.....	2
1.1.3 Intended Audience & Reading Suggestions...	2
1.1.4 Project Scope.....	2
1.2 Overall Description.....	2
1.2.1 Product Perspective.....	2
1.2.2 Product Features.....	2
1.2.3 User Classes & Characteristics.....	3
1.2.4 Operating Environment.....	3
1.2.5 Design & Implementation Constraints.....	4
1.2.6 User Documentation.....	4
1.2.7 Assumption & Dependencies.....	4
1.3 DFD.....	5

1.4DD.....	6
1.5System Features.....	6
1.6External Interface Requirements.....	9
1.6.1 User Interface	9
1.6.2 Hardware Interface.....	9
1.6.3 Software Interface.....	9
1.6.4 Communications Interface.....	10
1.7Other Non-functional Requirements.....	10
1.7.1 Performance Requirements.....	10
1.7.2 Safety Requirements.....	10
1.7.3 Software Quality Attributes	10
1.7.4 Security Requirements	11

CHAPTER 2 Estimation And Scheduling 12

2.1Size Estimation.....	12
2.2Cost Estimation.....	16
2.3.Gantt Chart.....	20

CHAPTER 3 Architectural Design..... 23

CHAPTER-1

SOFTWARE REQUIREMENTS AND SPECIFICATIONS

Introduction-:

Purpose

This software will help the company to be more efficient in registration of their patients and manage appointments, records of patients. It enables doctors and admin to view and modify appointments schedules if required. The purpose of this project is to computerize all details regarding patient details and hospital details.

Document Conventions

This document features some terminology which readers may be unfamiliar with

SRS: Software Requirement Specification

Front End: This stands for the JSP pages that a user will see when (s) he will access the application through the web.

Vehicle: Transport entity used for transportation of passengers.

Operator: A user which is hired for data entry of this system.

Entry : A data unit stored to the database.

DESC : Description

RAT : Rational

DEP : Dependency

Sign up: Creating New User

Log in : Logging in Existing User

Intended Audience and Reading Suggestions

The intended readers of this document are current and future developers working on “HOTEL MANAGEMENT PROJECT” and the sponsors of the project.

Product Scope

The system will be used as the application that serves hospitals, clinic, dispensaries or other health institutions. The intentions of the system is to increase the number of patients that can be treated and managed properly.

If the hospital management system is file based, management of the hospital has to put much effort on securing the files. They can be easily damaged by fire, insects and natural disasters. Also could be misplaced by losing data and information.

Product Perspective

This Hospital Patient Info Management System is a self-contained system that manages activities of the hospital.

Due to improperly managed details medical center faces quite a lot of difficulties in accessing past data as well as managing present data. The fully functional automated hospital management system which will be developed through this project will eliminate the disadvantages caused by the manual system by improving the reliability, efficiency and performance. The usage of a database to store patient, employee, stock details etc. will accommodate easy access, retrieval, search and manipulation of data. The access limitations provided through access privilege levels will enhance the security of the system. The system will facilitate concurrent access and convenient management of activities of the medical center.

Product Functions performed:

- Provide access to registered users only
- Registration of new patients
- Enable patient to view their record
- Enable patient to update their record
- Generate appointment date and timing
- Confirmation by doctor
- Modification in schedule by doctor
- Admin access to patient's record
- Admin access to doctor's record

USER CLASSES AND CHARACTERISTICS

Admin

Admin has the full access to the system which means he is able to manage any activity with regard to the system. He is the highest privileged user who can access to the system.

Key functions:

- Access and modify patient record
- Add new doctor entry in system database

Patient

Patients can choose the best preferred appointments from the options provided and can also change the appointment schedule or cancel it. Patients have access to only their records.

Key functions:

- Fix appointment
- Update schedule
- Cancel appointment

Doctor

Doctors can view the patient appointment list and provide the confirmation or make changes in the appointment list if required. Doctors have access to only records of those patients whom they are treating.

Key functions:

- Confirmation of appointment
- Modification of appointment list

Operating Environment

Software requirements

- Windows 7 or above operating system
- JRE 1.8
- MySQL server

Hardware Requirements

- Core i5 processor
- 2GB Ram
- 20GB of hard disk space in terminal machines
- 1TB hard disk space in Server Machine

Design and Implementation Constraints

- System is wirelessly networked with an encryption
- System is only accessible within the hospital's website only.
- Database is password protected.
- Should use less RAM and processing power.
- Each user should have individual ID and password.
- Only administrator can access the whole system.

User Documentation

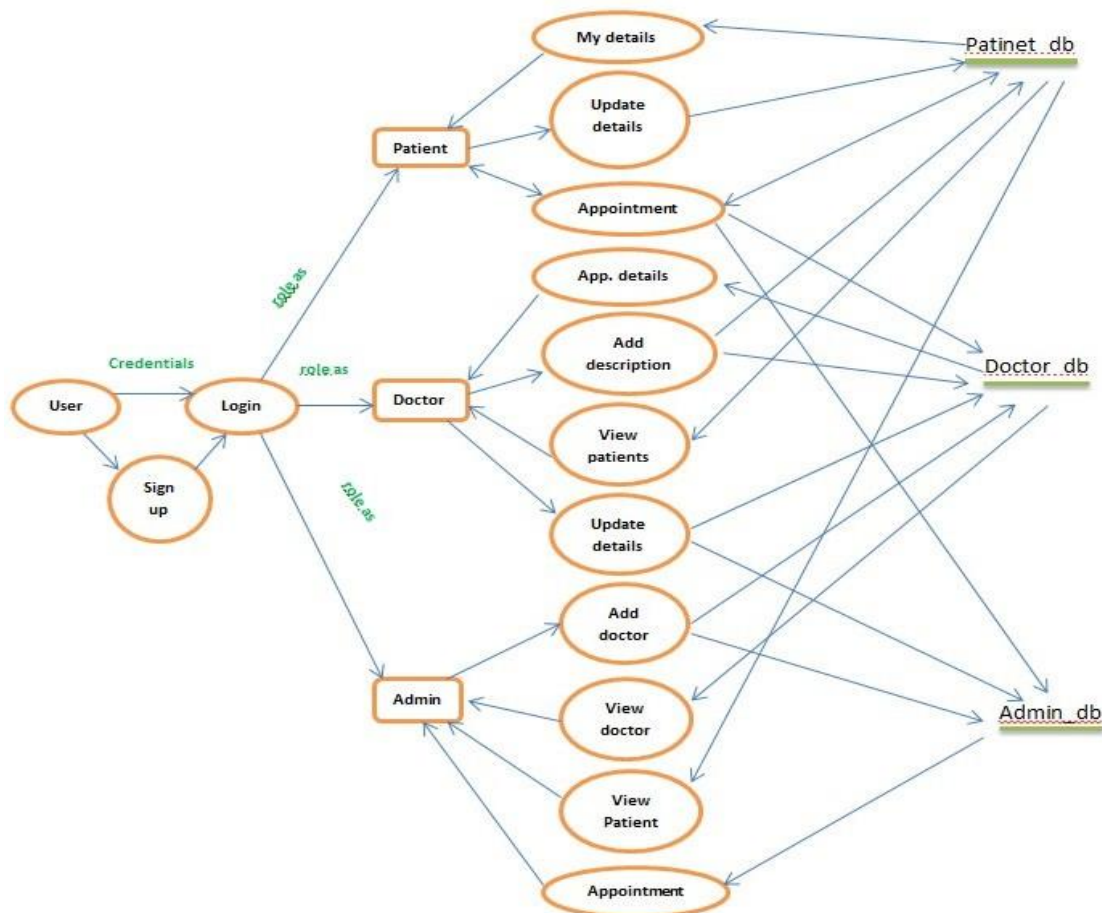
As a part of the system itself a user documentation is provided to the customers which gives an overview of the system. It will include the full description about the product and complete orderly followed steps to install the software. The users will get the opportunity to use the system without having any trouble. The user manual will include the email addresses to contact us in need. Tasks are listed alphabetically or logically grouped often using cross referenced indexes which helps the users to know exactly what sort of information they are looking for.

Assumptions and Dependencies

- Each user must have a valid user id and password
- Server must be running for the system to function
- Users must log in to the system to access any record.
- Only the Administrator can delete records.

DATA FLOW DIAGRAM

Data flow diagrams (also called **data flow graphs**) are commonly used during problem analysis. Data flow diagrams (DFDs) are quite general and are not limited to problem analysis for software requirements specification. They were in use long before the software engineering discipline began. DFDs are very useful in understanding a system and can be effectively used during analysis. A DFD shows the flow of data through a system. It views a system as a function that transforms the inputs into desired outputs. Any complex system will not perform this transformation in a "single step," and a data will typically undergo a series of transformations before it becomes the output. The DFD aims to capture the transformations that take place within a system to the input data so that eventually the output data is produced. The agent that performs the transformation of data from one state to another is called a process (or a bubble). So, a DFD shows the movement of data through the different transformations or processes in the system. The processes are shown by named circles and data flows are represented by named arrows entering or leaving the bubbles. A rectangle represents a source or sink and is a net originator or consumer of data. It should be pointed out that a DFD is not a flowchart. A DFD represents the flow of data, while a flowchart shows the flow of control. A DFD does not represent procedural information.



Data Dictionary

- Data dictionary is a collection of data that are used as a part of the system.
- It is a record of data about data.

Table name- Patient

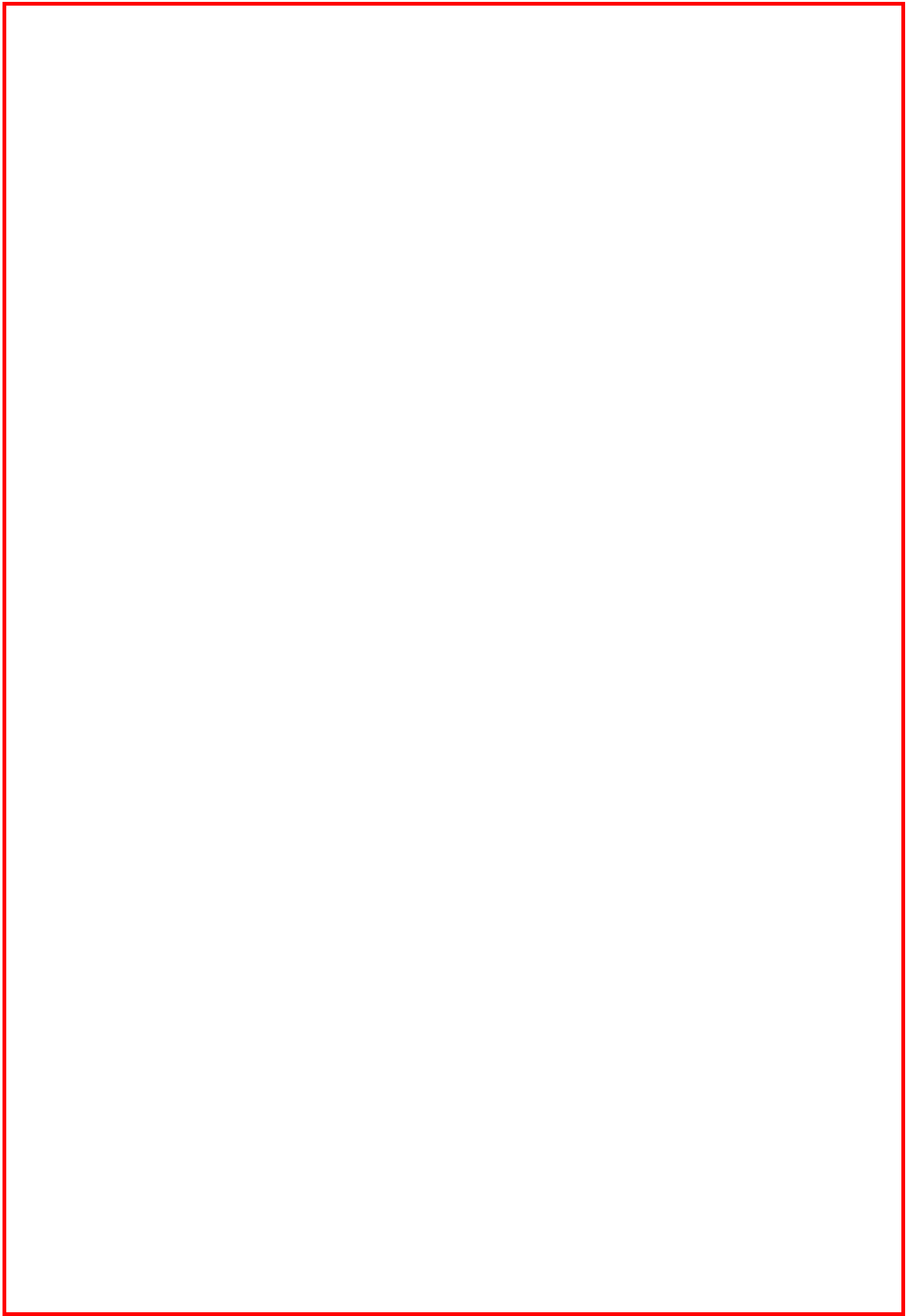
Primary Key- UID

Patient table				
Fields	Data type	Size	Constraints	Description
UID	int	2	primary key	this is patient Id
name	varchar	20	-	Name of patient
mobile	varchar	10	-	mobile no of patient
sex	varchar	1	-	sex of patient
email	varchar	20	-	email of patient
disease	varchar	20	-	disease from which he/she is suffering

Table name- Doctor

Doctor Table				
Fields	Data type	Size	Contraints	Description
DId	int	2	primary key	it is doctor id
Name	varchar	20	-	name of doctor
mobile	varchar	10	-	mobile no of doctor
speciality	varchar	10	-	speciality of doctor

This table keeps record of each doctor of hospital.



System Features

PATIENT

REGISTRATION

DESCRIPTION- The new patient can register themselves and add their details like name, age sex etc. The patient entry will be made in the patient database.

PRE -CONDITION – The patient must be a new patient.

MAIN FLOW OF EVENTS- 1. Patient selects sign up in login module.
2. A registration form get displayed
3. Patient fills the required details.

EXTENSIONS- if necessary fields left by user then prompt user to fill the necessary fields.

POST CONDITIONS- Patient record is added to patient database.

UPDATION

DESCRIPTION- The patient should be enabled to update his/her details and the changes should reflect in patient database.

PRE-CONDITION – The patient must be a registered patient.

MAIN FLOW OF EVENTS- 1. Patient logs in to the system.
2. Patient view his record
3. Patient selects update details.
4. Now patient may change the necessary fields.
5. Pop of update details.

EXTENSIONS – The patient cannot update details after treatment starts.

POST CONDITION- The record of patient is updated in patient database.

APPOINTMENT

DESCRIPTION- It shows users a list of available doctors, timings, dates and enables patients to select the most suitable appointment date and doctor. The patient may also the cancel the appointment if already available.

PRE-CONDITION- The patient must be a registered patient.

MAIN FLOW OF EVENTS – 1.patient first logs in to system.

2.View his record

3.create a new appointment or cancel the appointment.

EXTENSIONS-Patient can fix only one appointment for a particular department.

POST CONDITIONS- patient details are displayed and a new appointment is fix or a existing appointment is cancelled. The patient database is updated

DOCTOR

DESCRIPTION- The doctor view patient record/ update his details and add description of the treatment given to patient.

PRE-CONDITION – The doctor must be a registered doctor.

MAIN FLOW OF EVENTS – 1.Doctor logs in to the system.

2. Doctor may select view patient.

2.1 patient record is displayed with treatment history.

3 Doctor add description of patient treatment.

4.Doctor update his/her details.

EXTENSIONS- System does not allow the doctor to modify the qualification, hospital managed details.

POST CONDITION – The patient and doctor 's database are updated.

ADMIN

DESCRIPTION- The admin add doctor, view patient record and appointments

PRE -CONDITION- Admin must first log in with his/her credentials.

MAIN FLOW OF EVENTS- 1. Admin logs in the system.

2.admin may add doctor new doctor.

2.1 admin fills the doctor's details

3.admin view patient record.

- 3.1 admin enters the patient id in the system.
- 3.2 patient details are displayed.
- 4. admin view appointments for the current date.
 - 4.1 appointments for current date are displayed.

POST CONDITION- The doctor and admin database are updated.

External Interface Requirements

User Interfaces

This section provides a detailed description of all inputs into and outputs from the system. It also gives a description of the hardware, software and communication interfaces and provides basic prototypes of the user interface.

The protocol used shall be HTTP.

The Port number used will be 80.

There shall be logical address of the system in IPv4 format.

Hardware Interfaces

Laptop/Desktop PC

Purpose of this is to give information when Patients ask information about doctors, medicine available lab tests etc. To perform such Action it need very efficient computer otherwise due to that reason patients have to wait for a long time to get what they ask for.

Laser Printer (B/W)

Simply this device is for printing patients info etc.

Wi-Fi router

Wi-Fi router is used to for internetwork operations inside of a hospital and simply data transmission from pc's to sever.

Software Interfaces

- JDK 1.8 - Java is fast, secure, and reliable. From laptops to data centers, game consoles to scientific supercomputers, cell phones to the Internet,
- Netbeans 8.1 - IDE for Java developing.
- MySQL server - Database connectivity and management
- OS Windows 7/8/8.1- Very user friendly and common OS
- JRE 1.8 - JAVA Runtime Environment for run Java Application and System
- MySQL server - Database connectivity

Communications Interfaces

- NIC (Network Interface Card) – It is a computer hardware component that allows a computer to connect to a network . NICs may be used for both wired and wireless connections.
- CAT 5 network cable- for high signal integrity
- TCP/IP protocol- Internet service provider to access and share information over the Internet
- Ethernet Communications Interface- Ethernet is a frame-based computer network technology for local area networks (LANs)
- Ubiquitous, easy to set up and easy to use. Low cost and high data transmission rates

Other Nonfunctional Requirements

Performance Requirements

Response time-The system will give responses within 1 second after checking the patient information and other information.

Capacity-The system must support 1000 people at a time

User interface- User interface screen will response within 5 seconds

Safety Requirements

If there is extensive damage to a wide portion of the database due to catastrophic failure, such as a disk crash, the recovery method restores a past copy of the database that was backed upto archival storage and reconstructs a more current state by reapplying or redoing the operations of committed transactions from the backed up log, up to the time of failure. All the administrative and data entry operators have unique logins so system can understand who is login in to system right now no intruders allowed except system administrative nobody cannot change record and valuable data.

Software Quality Attributes

AVAILABILITY: The system shall be available all the time.

CORRECTNESS: A bug free software which fulfill the correct need/requirements of the client.

MAINTAINABILITY: The ability to maintain ,modify information and update fix problems of the system

USABILITY: software can be used again and again without distortion.

ACCESSIBILITY: Administrator and many other users can access the system but the access level is controlled for each user according to their work scope.

ACCURACY: The reliability on the information/output. Can depend/be sure of the outcome.

STABILITY:The system outcome/output won't change time to time. Same output
Will be given always for a given input.

Security Requirements

- 1.Want take the responsibility of failures due to hardware malfunctioning.
- 2.Warranty period of maintaining the software would be one year.
- 3.Additional payments will be analysed and charged for further maintenance
- 4.If any error occur due to a user's improper use. Warranty will not be allocated to it.
- 5.No money back returns for the software.

CHAPTER-2

ESTIMATION AND SCHEDULING

SIZE ESTIMATION

Function-Based Metrics

The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.⁴ Using historical data, the FP metric can then be used to (1) estimate the cost or effort required to design, code, and test the software; (2) predict the number of errors that will be encountered during testing; and (3) forecast the number of components and/or the number of projected source lines in the implemented system. Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and qualitative assessments of software complexity. Information domain values are defined in the following manner:

Number of external inputs (EIs). Each external input originates from a user or is transmitted from another application and provides distinct application-oriented data or control information. Inputs are often used to update internal logical files (ILFs). Inputs should be distinguished from inquiries, which are counted separately.

Number of external outputs (EOs). Each external output is derived data within the application that provides information to the user. In this context external output refers to reports, screens, error messages, etc. Individual data items within a report are not counted separately.

Number of external inquiries (EQs). An external inquiry is defined as an online input that results in the generation of some immediate software response in the form of an online output (often retrieved from an ILF).

Number of internal logical files (ILFs). Each internal logical file is a logical grouping of data that resides within the application's boundary and is maintained via external inputs.

Number of external interface files (EIFs). Each external interface file is a logical grouping of data that resides external to the application but provides information that may be of use to the application.

Once these data have been collected, the table in Figure 23.1 is completed and a complexity value is associated with each count. Organizations that use function point methods develop criteria for determining whether a particular entry is simple, average, or complex. Nonetheless, the determination of complexity is somewhat subjective.

To compute function points (FP), the following relationship is used:

$$FP = \text{count total} \times [0.65 + 0.01 \times \sum (F_i)]$$

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total							

SIZE ESTIMATION FOR THIS PROJECT

FTR's	DATA ELEMENTS		
	1-4	5-15	> 15
0-1	Low	Low	Ave
2	Low	Ave	High
3 or more	Ave	High	High

EXTERNAL INPUTS

1 LOGIN MODULE

1.1 USERNAME	DATA ELEMENTS	FILE TYPE REFERENCE	COMPLEXITY
	2	1	SIMPLE
1.2 PASSWORD			

2. REGISTRATION MODULE

2.1 NAME			
2.2 DOB			
2.3 SEX	DATA ELEMENTS	FILE TYPE REFERNCED	COMPLEXITY
2.4 EMAIL	7	2	AVG
2.5 BLOOD TYPE			
2.6 MOBILE NO			
2.7 ADDRESS			

3.APPOINTMENT

3.1 CATEGORY	DATA ELEMENTS	FILE TYPE REFERNCED	COMPLEXITY
3.2 DATE	3	3	HIGH
3.3 TIME			

4.ADD DESCRIPTION

4.1 TREATMENT	DATA ELEMENTS	FILE TYPE REFERNCED	COMPELXITY
4.2 RATE	2	2	LOW

5.ADD DOCTOR

5.1 NAME			
5.2 AGE	DATA ELEMENTS	FILE TYPE REFERNCED	COMPLEXITY
5.3 SEX	6	2	AVG
5.4 DOB			
5.5 DATE OF JOIN			
5.6 QUALIFICATION			

TOTAL EXTERNAL INUPUTS = 18

- 7 = **AVG COMPLEXITY**
- 3=**HIGH COMPLEXITY**
- 2=**LOW COMPLEXITY**
- 5=**AVG COMPLEXITY**

EXTERNAL OUTPUTS

PATIENT

- 1.1 MY DETAILS
- 1.2 VIEW BOOKING

DOCTOR

- 2.1 APPOINTMENT DETAILS
- 2.2 VIEW PATIENT

ADMIN

- 3.1 VIEW DOCTOR
- 3.2 VIEW PATIENT
- 3.3 VIEW APPOINTMENTS

ALL HAVE SIMPLE COMPLEXITY

TOTAL EXTERNAL OUTPUTS = 7

LOGICAL INTERNAL FILES

- 1. PATIENT FILE
- 2. DOCTOR FILE
- 3. ADMIN FILE
- 4. LOGIN FILE

COMPLEXITY IS SIMPLE FOR ALL LOGICAL INTERNAL FILES

TOTAL LOGICAL INTERNAL FILES=3

EXTERNAL INTERFACE FILES =0

EXTERNAL INQUIRIES

PATIENT- CHECK APPOINTMENT DETAILS
ADMIN- APPOINTMENT FOR CURRENT DATE

COMPLEXITY IS SIMPLE FOR ALL EXTERNAL INQUIRIES

TOTAL EXTERNAL INQUIRIES =2

$$\text{UFP} = (7*4+3*6+2*3+5*6) +7*4 +4*7+2*3$$

$$\text{UFP}=144$$

Considering all adjustment factors have average influence

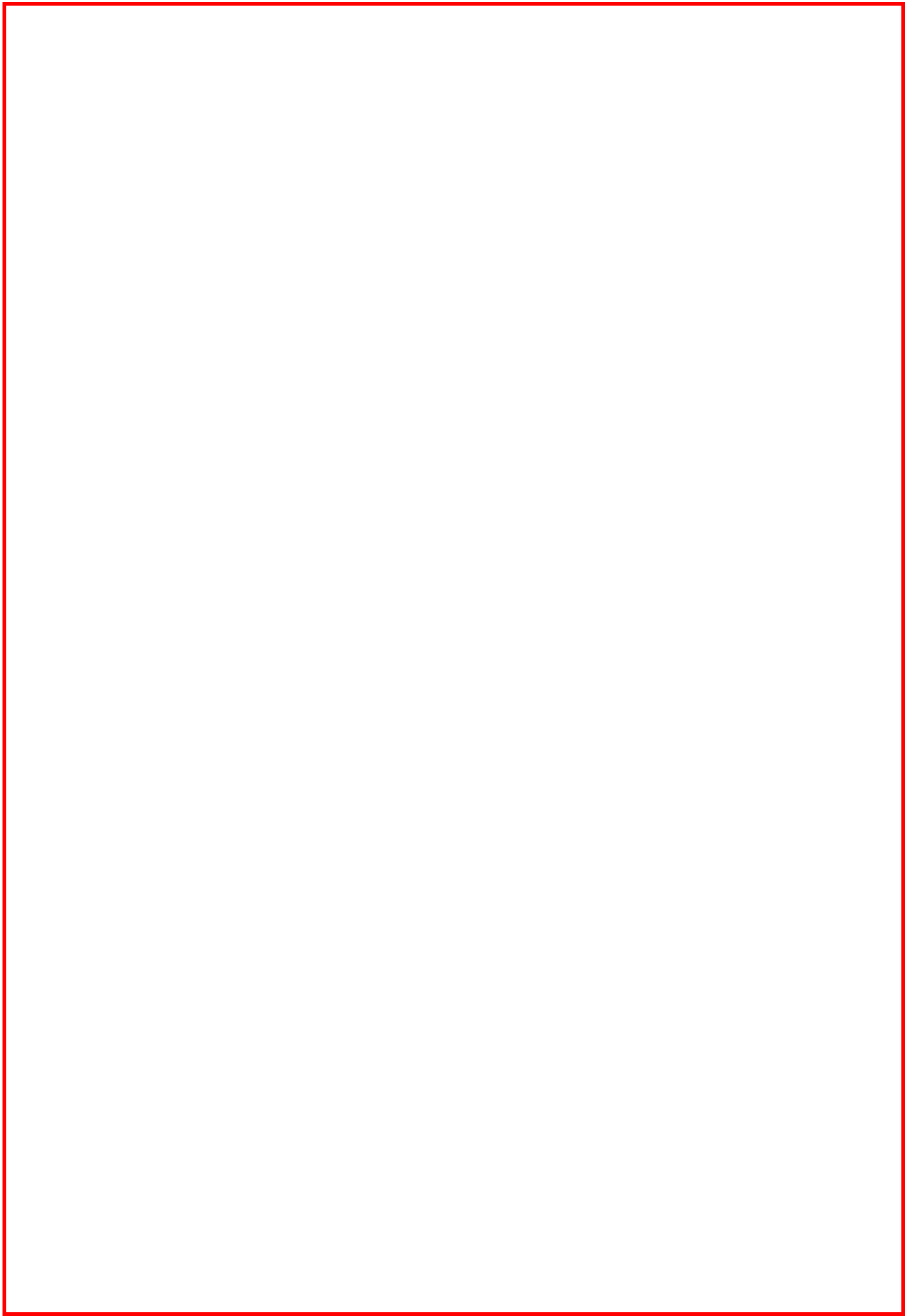
$$\text{CAF} =0.65+0.01*14*3$$

$$\text{CAF}=1.07$$

$$\text{FP}= 144*1.07$$

$$\text{FP}=154.08=155$$

SO FUNCTION POINT COUNT=155



Cost Estimation

The COCOMO II Model

In his classic book on “software engineering economics,” Barry Boehm [Boe81] introduced a hierarchy of software estimation models bearing the name COCOMO, for Constructive Cost Model. The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry. It has evolved into a more comprehensive estimation model, called COCOMOII [Boe00]. Like its predecessor, COCOMO II is actually a hierarchy of estimation models that address the following areas:

- Application composition model. Used during the early stages of software engineering, when prototyping of user interfaces, consideration of software and system interaction, assessment of performance, and evaluation of technology maturity are paramount.
- Early design stage model. Used once requirements have been stabilized and basic software architecture has been established.
- Post-architecture-stage model. Used during the construction of the software.

Like all estimation models for software, the COCOMO II models require sizing information. Three different sizing options are available as part of the model hierarchy: object points, function points, and lines of source code.

Object type	Complexity weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL component			10

The COCOMO II application composition model uses object points and is illustrated in the following paragraphs. It should be noted that other, more sophisticated estimation models (using FP and KLOC) are also available as part of COCOMO II. Like function points, the object point is an indirect software measure that is computed using counts of the number of (1) screens (at the user interface), (2) reports, and (3) components likely to be required to build the application. Each object instance (e.g., a screen or report) is classified into one of three complexity levels (i.e., simple, medium, or difficult) using criteria suggested by Boehm [Boe96]. In essence, complexity is a function of the number and source of the client and server data tables that are required to generate the screen or report and the number of views or sections presented as part of the screen or report.

Once complexity is determined, the number of screens, reports, and components are weighted according to the table illustrated in Figure . The object point count is then determined by multiplying the original number of object instances by the weighting factor in the figure and summing to obtain a total object point count. When component-based development or general software reuse is to be applied, the percent of reuse (%reuse) is estimated and the object point count is adjusted:

$$NOP = (\text{object points}) \times [(100 - \%reuse)/100]$$

where NOP is defined as new object points.

To derive an estimate of effort based on the computed NOP value, a “productivity rate” must be derived. Figure B presents the productivity rate

$$\text{PROD} = \frac{\text{NOP}}{\text{person-month}}$$

for different levels of developer experience and development environment maturity. Once the productivity rate has been determined, an estimate of project effort is computed using

$$\text{Estimated effort} = \frac{\text{NOP}}{\text{PROD}}$$

In more advanced COCOMO II models, a variety of scale factors, cost drivers, and adjustment procedures are required.

PRODUCTIVITY RATE FOR OBJECT POINT COUNTS

Developer's experience/capability	Very low	Low	Nominal	High	Very high
Environment maturity/capability	Very low	Low	Nominal	High	Very high
PROD	4	7	13	25	50

COST ESTIMATION FOR THIS PROJECT

SCREENS

- 1 LOGIN TYPE
2. LOGIN
3. REGISTRATION
4. MY DETAILS
5. UPDATE DETAILS
- 6.APPOINTMENT DETAILS
7. BOOK APPOINTMENT
8. VIEW BOOKINGS
9. CANCEL BOOKING
- 10.APPOINTMENT FOR DOCTOR
11. VIEW PATIENT

13. ADD DESCRIPTION
14. UPDATE DOCTOR DETAILS
15. ADD DOCTOR
16. VIEW PATIENT BY ADMIN
17. VIEW APPOINTMENTS FOR DATE
- 18. VIEW DOCTOR**

TOTAL SCREENS= 18

REPORTS

1. REGISTERED SUCCESSFULLY
2. DETAILS SUCCESSFULLY UPDATED
3. APPOINTMENT SUCCESSFULLY MADE
4. BOOKING SUCCESSFULLY CANCELLED
5. DOCTOR ENTRY MADE

TOTAL REPORTS=5

3 GL MODULES

1. NETBEANS
- TOTAL 3GL MODULES =1

CONSIDERING ALL OF ABOVE HAVE AVERAGE COMPEXITY WE GET

OBJECT POINT= $18*2+5*5+10*1$
OP=71

NOP=OBJECT POINT COUNT $-(1- \%REUSE/100)$
 $NOP=71*(1-0.30)$
NOP=49.7

NOW TAKING THE DEVELOPER EXPERIENCE AND SKILLS AS NOMINAL WE HAVE
PRODUCTIVITY=13

EFFORT= NOP/PRODUCTIVITY

EFFORT= $49.7/13$
EFFORT=3.8

EFFORT= 4 PERSON/MONTH

SCHEDULING

Scheduling of a software project does not differ greatly from scheduling of any multitask engineering effort. Therefore, generalized project scheduling tools and techniques can be applied with little modification for software projects.

Program evaluation and review technique (PERT) and the critical path method (CPM) are two project scheduling methods that can be applied to software development. Both techniques are driven by information already developed in earlier project planning activities: estimates of effort, a decomposition of the product function, the selection of the appropriate process model and task set, and decomposition of the tasks that are selected.

Interdependencies among tasks may be defined using a task network. Tasks, sometimes called the project work breakdown structure (WBS), are defined for the product as a whole or for individual functions.

Both PERT and CPM provide quantitative tools that allow us to (1) determine the critical path—the chain of tasks that determines the duration of the project, (2) establish “most likely” time estimates for individual tasks by applying statistical models, and (3) calculate “boundary times” that define a time “window” for a particular task.

Gantt Chart

When creating a software project schedule, you begin with a set of tasks (the work breakdown structure). If automated tools are used, the work breakdown is input as a task network or task outline. Effort, duration, and start date are then input for each task. In addition, tasks may be assigned to specific individuals.

As a consequence of this input, a time-line chart, also called a Gantt chart, is generated. A time-line chart can be developed for the entire project. Alternatively, separate charts can be developed for each project function or for each individual working on the project.

Figure a illustrates the format of a time-line chart. It depicts a part of a software project schedule that emphasizes the concept scoping task for a word-processing (WP) software product. All project tasks (for concept scoping) are listed in the lefthand column. The horizontal bars indicate the duration of each task. When multiple bars occur at the same time on the calendar, task concurrency is implied. The diamonds indicate milestones.

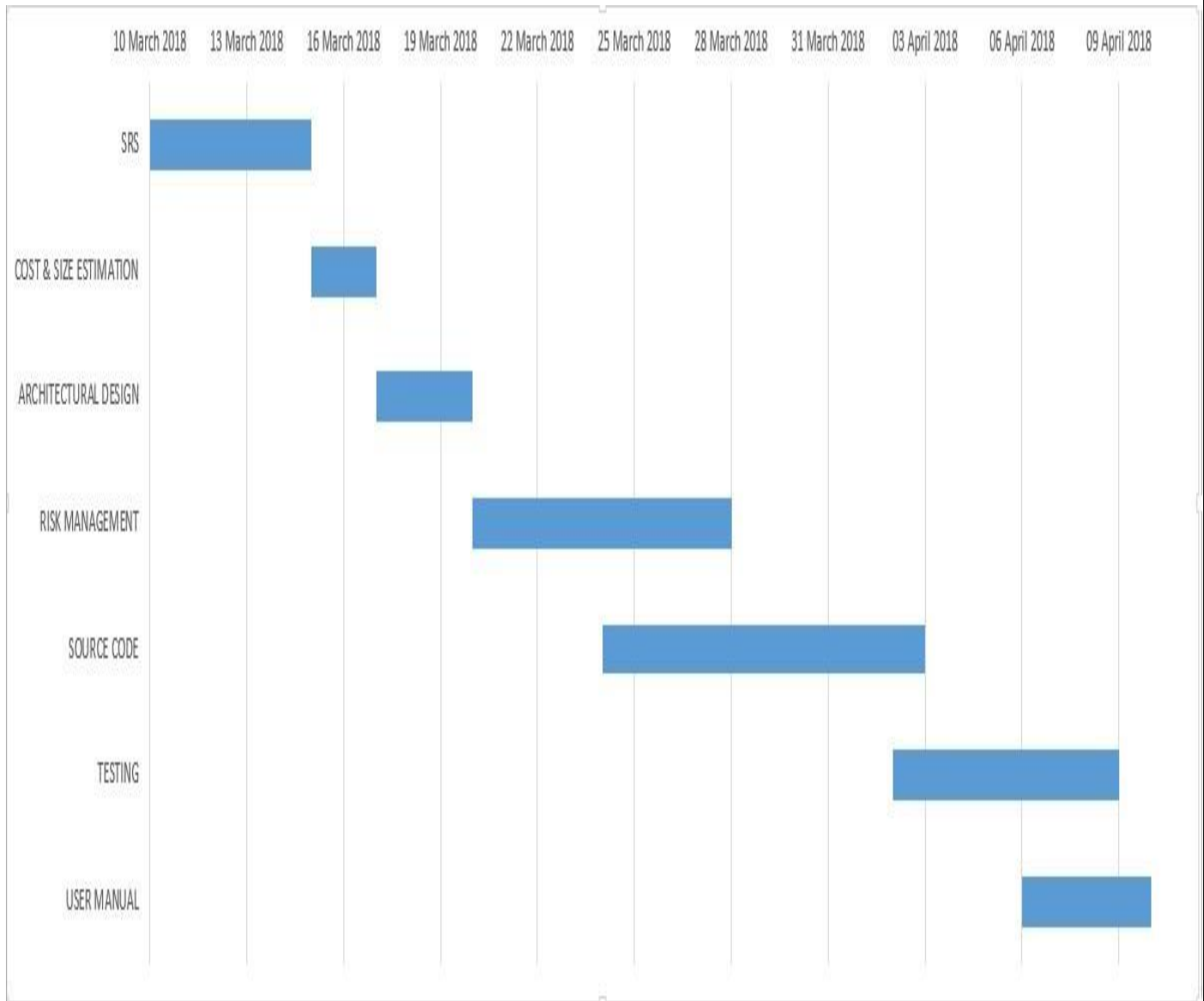
An example of a gantt chart is:

3. Scheduling

3.1 Schedule table

WORK TASKS	SCHEDULE
1. IDENTIFY NEEDS AND BENEFITS	
Meet with concerned members	Week 1
Identify needs and project constraints	Week 1
Establish problem statement	Week 2
Milestone	Week 2
2. REQUIREMENT ANALYSIS	
Detailed discussion of the project	Week 3
Creating Data flow Diagram	Week 4
Data Dictionary	Week 5
Milestone	Week 5
3. PROJECT MANAGEMENT	
Computing F.P. and Effort	Week 5
Schedule table	Week 6
Risk table	Week 7
Timeline Chart	Week 8
Milestone	Week 8
4. DESIGN ENGINEERING	
Architectural Design	Week 8
Data Design	Week 9
Pseudo Code	Week 9
Milestone	Week 10
5. TESTING	Week 12

GANTT CHART FOR OUR HOSPITAL MANAGEMENT SYSTEM



CHAPTER-3

Architectural design

Requirements of the software should be transformed into an architecture that describes the software's top-level structure and identifies its components. This is accomplished through architectural design (also called **system design**), which acts as a preliminary 'blueprint' from which software can be developed. **IEEE** defines architectural design as 'the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a [computer](#) system.' This framework is established by examining the software requirements document and designing a model for providing implementation details. These details are used to specify the components of the system along with their inputs, outputs, functions, and the interaction between them. An architectural design performs the following functions.

It defines an abstraction level at which the designers can specify the functional and performance behaviour of the system.

2. It acts as a guideline for enhancing the system (when ever required) by describing those features of the system that can be modified easily without affecting the system integrity.
3. It evaluates all top-level designs.
4. It develops and documents top-level design for the external and internal interfaces.
5. It develops preliminary versions of user documentation.
6. It defines and documents preliminary test requirements and the schedule for software integration.

Though the architectural design is the responsibility of developers, some other people like user representatives, systems engineers, hardware engineers, and operations personnel are also involved. All these stakeholders must also be consulted while reviewing the architectural design in order to minimize the risks and errors.

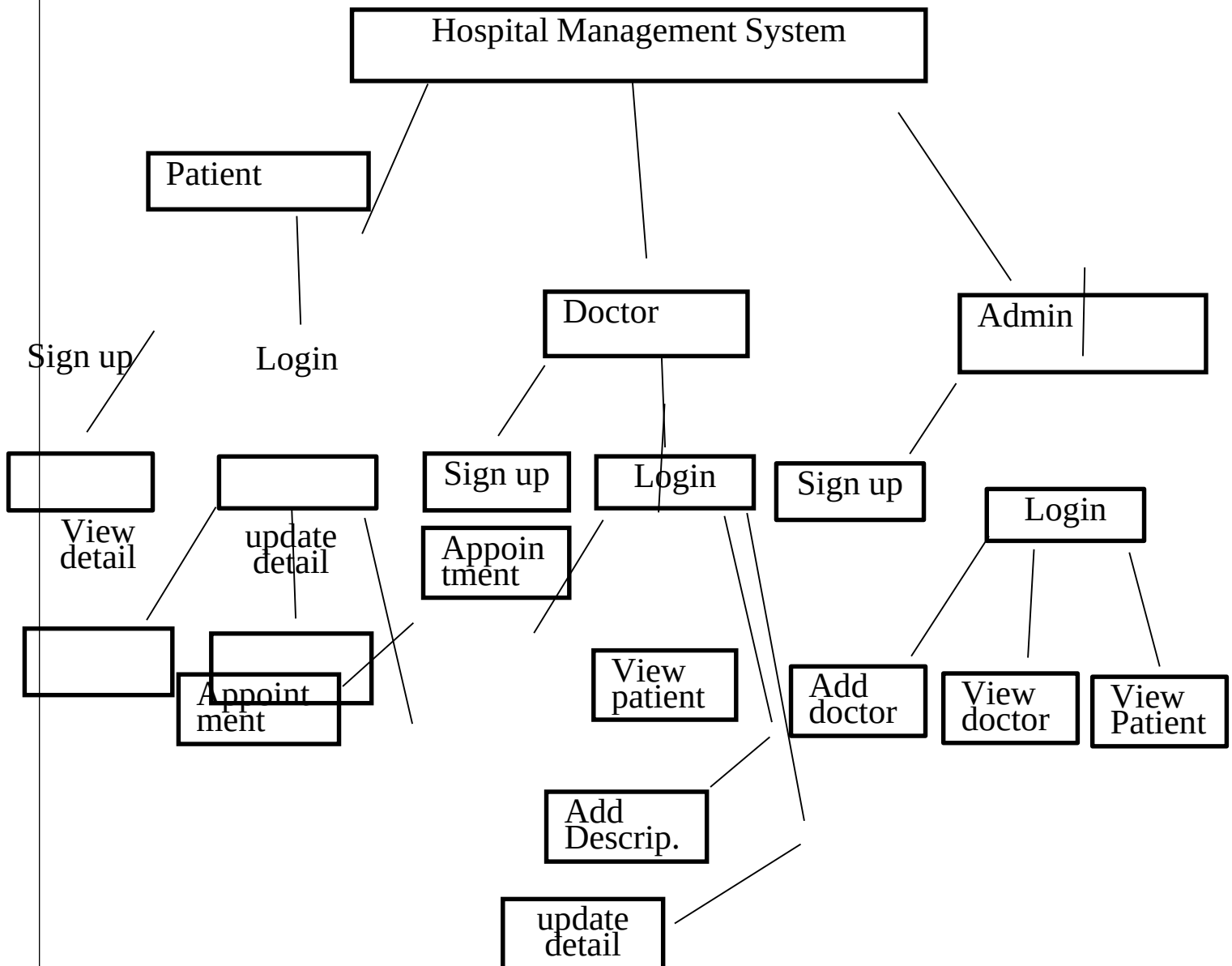
Architectural Design Representation

Architectural design can be represented using the following models.

1. **Structural model:** Illustrates architecture as an ordered collection of program components
2. **Dynamic model:** Specifies the behavioral aspect of the software architecture and indicates how the structure or system configuration changes as the function changes due to change in the external environment
3. **Process model:** Focuses on the design of the business or technical process, which must be implemented in the system

4. **Functional model:** Represents the functional hierarchy of a system
5. **Framework model:** Attempts to identify repeatable architectural design patterns encountered in similar types of application. This leads to an increase in the level of abstraction.

0

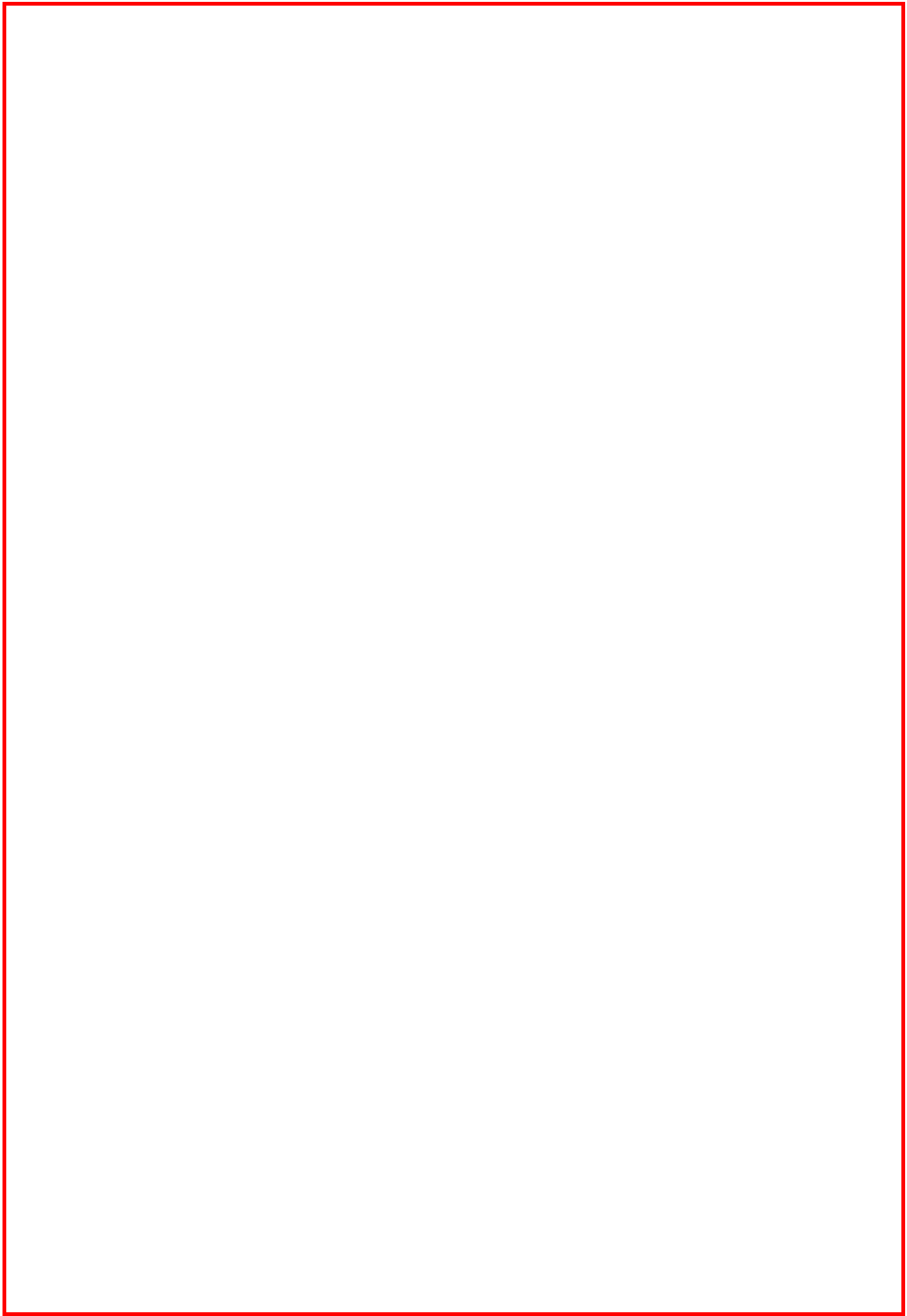


CHAPTER-4

RISK ANALYSIS

RISK ITEMS THEIR CATEGORIES,IMPACTS AND THEIR MITIGATION PLAN

S.NO	RISK	CATEGORY	PROBABILITY(P)	IMPACT(I)	EXPOSURE $E=P*I$	RMMM PLAN
1	SOME TEAM MEMBERS LEAVE THE PROJECT IN BETWEEN	TECHNICAL RISK	20%	2	0.4	USE BACKUP STAFFS WHO KNOWS WHAT WAS GOING ON PROJECT
2	DELIEVERY DEADLINE TIGHTENED	PROJECT RISK	30%	1	0.3	TEAM MAY USE EXTRA MEMBERS TO DO JOB ON SCHEDULED TIME
3	LOSING OF ALL PROJECT DATA THIS MAY HAPPEN DUE TO HARD DISK FAILURE	PROJECT RISK	20%	2	0.2	CARRY OUT BACKUP OF ESSENTIAL DATABASES,SOURCE CODE ETC.
4	TEAM DISTENSION/LACK OF COHESION	PROJECT RISK	10%	3	0.3	WE MAKE SOME RULES ON HOW WE CONSULT EACH OTHER



CHAPTER-5

IMPLEMENTATION

Module screens

LOGIN PAGE



The screenshot displays the login interface for the Hospital Management System. At the top, the title "Hospital Management System" is centered in a purple font. Below this, the labels "Login" and "Password" are positioned to the left of two white input fields. A blue "LOGIN" button is centered below the input fields. At the bottom, a red text prompt reads "NOT REGISTERED ..CLICK TO REGISTER...!!".

Hospital Management System

Login

Password

LOGIN

NOT REGISTERED ..CLICK TO REGISTER...!!

REGISTRATION PAGE

Hospital Management System

REGISTRATION

Name	
DOB	
Sex	
E-mail	
mobile no.	
Address	

Submit

PATIENT PAGE

My Details

Hospital Manangement System

Patient

My Details

Update Details

Appointment

Login

User ID

Name

Mobile no.

E-mail

Address

uid	Name	treatment	Date

Update Details

[My details](#) [Update details](#) [Appointment](#) [Logout](#)

Name

Email

Mobil...

Adress

Appointment

My details

Update details

Appointment

Logout

App Id

Categorise

Bone

▼

Date

Time

Book Appointment

View Booking

Cancel Booking

DOCTOR APPOINTMENT

Hospital Management System

Doctor

Appointment details

View Patient

Add description

Update my details

Logout

My Appointments

UID	Date	Time

DOCTOR VIEW PATIENT

Hospital Management System

Doctor

Appointment details

View Patient

Add description

Update my details

Logout

Patient details

PID

Search

UID	Name	Mobile	Sex	Email	Disease

DOCTOR ADD DESCRIPTION

Hospital Management System

Doctor

Appointment details

View Patient

Add description

Update my details

Logout

ADD Patient details

UID

Name

Treatment for :

Note

Submit

Java Coding

Login

```
try{
    String Username=jTextField1.getText();
    String pass= new String(jPasswordField2.getPassword());
    Class.forName("java.sql.Driver");
    Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/Airline","root","");
    Statement stmt= conn.createStatement();
    String query= "Select * from user where UserName='"+Username+"'";
    ResultSet rs= stmt.executeQuery(query);
    rs.next();
    String User= rs.getString("UserName");
    String P= rs.getString("Password");
    String type= rs.getString("UserType");
    if(pass.equals(P)){
        JFrame2.setVisible(true);
        this.setVisible(false);
    }
    else{
        JOptionPane.showMessageDialog(null, "Enter Correct Values .. ");
    }

    rs.close();
    stmt.close();
    conn.close();
}
catch (Exception e){
    JOptionPane.showMessageDialog(null, "Unable to Connect ...."+e);
}
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    JFrame14.setVisible(true);
    this.setVisible(false);
}

private void jButton36ActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
}
```

Registration

```

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    try{

        String name, dob,sex,email,mobileno, address;
        name= jTextField1.getText();
        dob= jTextField2.getText();
        sex= jTextField3.getText();
        email=jTextField4.getText();
        mobileno=jTextField5.getText();
        address= jTextArea1.getText()
        Class.forName("java.sql.Driver");
        Connection
        conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/hospital","root","");
        Statement stmt= conn.createStatement();
        String query= "Insert into patient
        values('"+name+"','"+dob+"','"+sex+"','"+email+"','"+mobileno+"','"+address+"');";
        stmt.executeUpdate(query);
        stmt.close();
        conn.close();
        JOptionPane.showMessageDialog(null, "Patient Record has been Saved ...");

        jTextField1.setText("");
        jTextField2.setText("");
        jTextField3.setText("");
        jTextField4.setText("");
        jTextField5.setText("");
        jTextArea1.setText("");
    }
    catch (Exception e){
        JOptionPane.showMessageDialog(null, "Unable to Save.... "+e);
    }
}

```

My Details

```

try
{

class.forName("java.sql.Driver");

connection con = DriverManager.getConnection("jdbc:mysql://localhost/test","root","xyz");

Statement stmt = con.createStatement();

String query= "update tablename set email = '"+tf2.getText() + "dmobile = " + mobiletf.getText()+"',
Address = '"+Addtf.getText()+"' where name = '"+nametf.getText()+"';";

stmt.executeUpdate(query);

```

```

Joption.show MessageDialog(null, "Record updated successfully")

}

catch (exception e)
{

Joptionpane.showmessageDialog(null, "error in updation!");
}

```

Update details

```

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    class GenerateId{
        int randomId=0;
        Random rand=new Random();
        for(int j=0;j<10;j++){
            randomId=rand.nextLong();
        }
        return randomId;
    }

    try{
        Class.forName("java.sql.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/Airline","root","");
        Statement stmt= conn.createStatement();
        int randomId;
        id= Integer.parseInt(jTextField7.getText());
        String query= "Select * from patient where id="+randomId+"";
        ResultSet rs;
        rs= stmt.executeQuery(query);
        String name, email, mobileno, address;
        rs.first();
        name= rs.getString("Name");
        email= rs.getString("email");
        mobileno= rs.getString("mobileno");
        address= rs.getString("Address");

        jTextField2.setText(name);
        jTextField4.setText(email);
        jTextField3.setText(mobileno);
        jTextField5.setText(address);
        rs.close();
        stmt.close();
        conn.close();
    }
    catch (Exception e){

```

```

        JOptionPane.showMessageDialog(null, "Unable to Search Patient.... "+e);
    }

}

private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    jFrame4.setVisible(true);
}

private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
}

```

Appointment

```

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    jFrame2.setVisible(true);
}

private void jTextField3ActionPerformed(java.awt.event.ActionEvent evt) {
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    jFrame3.setVisible(true);
}

private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
}

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
}

private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) {
    class GenerateId{
        int randomId=0;
        Random rand=new Random();
        for(int j=0;j<10;j++){
            randomId=rand.nextLong();
        }
        return randomId;
    }
    try{
        int randomId;
        String name, fname, phone, address;
        id= Integer.parseInt(jTextField3.getText());
        name= jTextField4.getText();
    }
}

```

```

        fname= jTextField5.getText();
        phone= jTextField6.getText();
        address= jTextArea1.getText();
        Class.forName("java.sql.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/Airline","root","");
        Statement stmt= conn.createStatement();
String query= "Insert into Appointment
values("+randomId+", '"+cateogry+"', '"+date+"', '"+time+"')";
        stmt.executeUpdate(query);
        stmt.close();
        conn.close();
        JOptionPane.showMessageDialog(null, "Appointment has been done .. ");

        jTextField1.setText("");
        jTextField4.setText("");
        jTextField3.setText("");

    }
    catch (Exception e){
        JOptionPane.showMessageDialog(null, "Unable to Save.... "+e);
    }
}

}

private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {
class GenerateId{
    int randomId=0;
    Random rand=new Random();
    for(int j=0;j<10;j++){
        randomId=rand.nextLong();
    }
    return randomId;
    try{
        Class.forName("java.sql.Driver");
        Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/Airline","root","");
        Statement stmt= conn.createStatement();
        int id;
        id= Integer.parseInt(jTextField7.getText());
        String query= "Select * from Appointment where ID="+randomId+"";
        ResultSet rs;
        rs= stmt.executeQuery(query);
        String cateogry,date,time;
        rs.first();
        cateogry=rs.getString("cateogry");
        date= rs.getString("date");
        time= rs.getString("time");

        jTextField1.setText(id);
        jTextField4.setText(date);

```

```

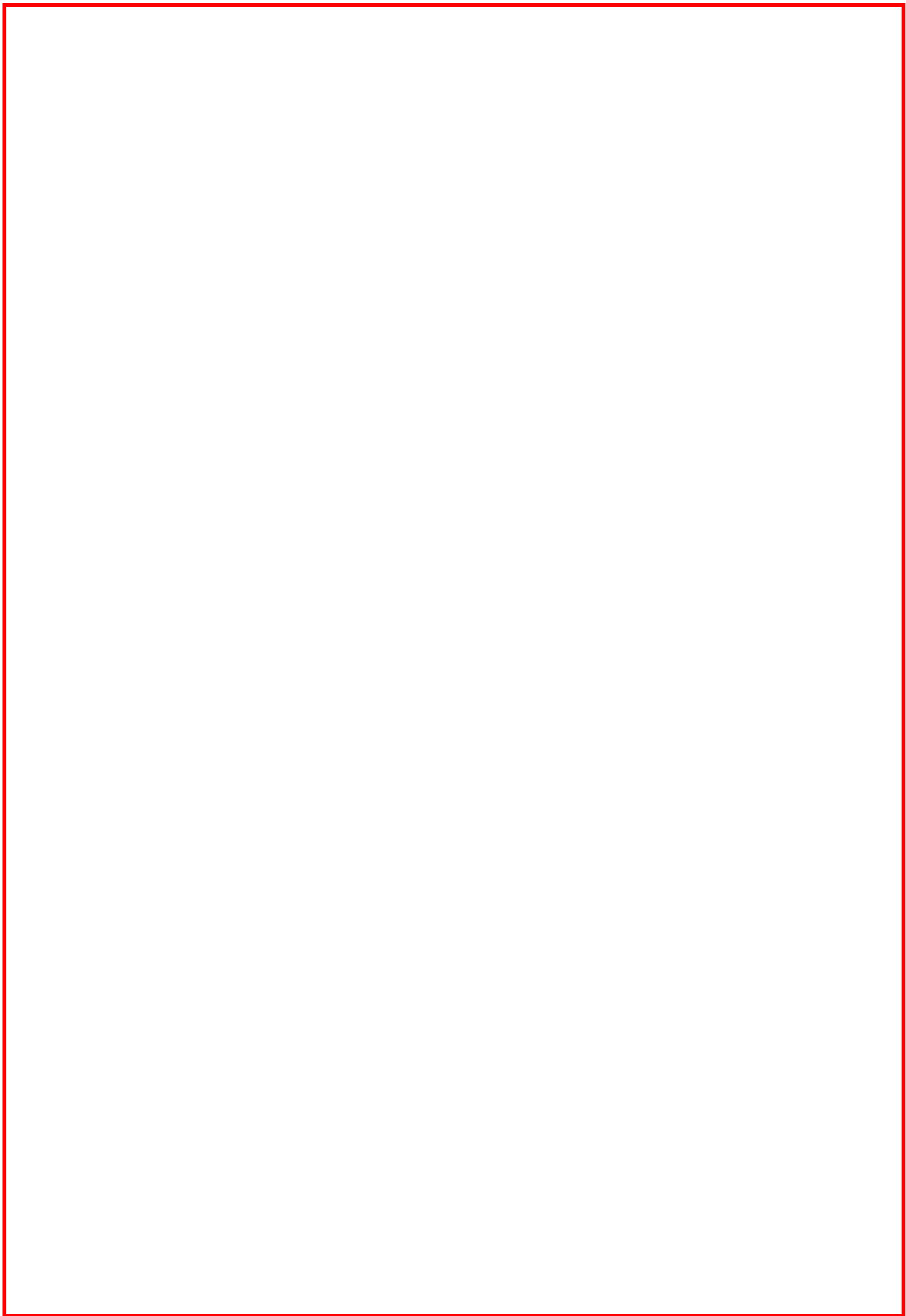
        jTextField3.setText(time);
        jComboBox1.setSelected(cateogry);
        rs.close();
        stmt.close();
        conn.close();
    }
    catch (Exception e){
        JOptionPane.showMessageDialog(null, "Unable to Search patient.... "+e);
    }

}

private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
class GenerateId{
    int randomId=0;
    Random rand=new Random();
    for(int j=0;j<10;j++){
        randomId=rand.nextLong();
    }
    return randomId;
    try{
        Class.forName("java.sql.Driver");
        Connection conn=DriverManager.getConnection
            ("jdbc:mysql://localhost:3306/Hospital","root","");
        Statement stmt= conn.createStatement();
        int id;
        id= Integer.parseInt(jTextField7.getText());
        String query= "Select * from Appointment where ID="+randomId+"";
        ResultSet rs;
        rs= stmt.executeQuery(query);
        rs.close();
        stmt.close();
        conn.close();
    }
    catch (Exception e){
        JOptionPane.showMessageDialog(null, "The Booking is to be cancel.... "+e);
    }

}
}

```



CHAPTER-6

TESTING

DOCTOR VIEW PATIENT MODULE

WHITE BOX TESTING

White-box testing, sometimes called glass-box testing, is a test-case design philosophy that uses the control structure described as part of component-level design to derive test cases. Using white-box testing methods, you can derive test cases that (1) guarantee that all independent paths within a module have been exercised at least once, (2) exercise all logical decisions on their true and false sides, (3) execute all loops at their boundaries and within their operational bounds, and (4) exercise internal data structures to ensure their validity.

The white box testing has another branch called basis path testing.

1)BASIS PATH TESTING

Basis path testing is a white-box testing technique first proposed by Tom McCabe [McC76]. The basis path method enables the test-case designer to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths. Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.

Flow Graph Notation

Before we consider the basis path method, a simple notation for the representation of control flow, called a flow graph (or program graph) must be introduced

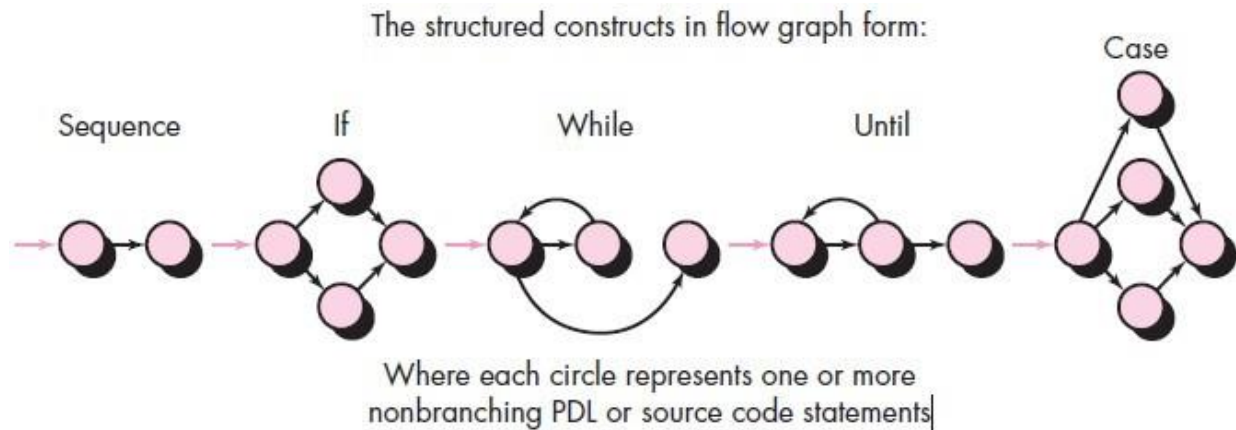


Fig.a

To illustrate the use of a flow graph, consider the procedural design representation in Above figure a. Here, a flowchart is used to depict program control structure. Figure b maps the flowchart into a corresponding flow graph (assuming that no compound conditions are contained in the decision diamonds of the flowchart). Referring to Figure b, each circle, called a flow graph node, represents one or more procedural statements. A sequence of process boxes and a decision diamond can map into a single node. The arrows on the flow graph, called edges or links, represent flow of control and are analogous to flowchart arrows. An edge must terminate at a node, even if the node does not represent any procedural statements (e.g., see the flow graph symbol for the if-then-else construct). Areas bounded by edges and nodes are called regions. When counting regions, we include the area outside the graph as a region.⁴

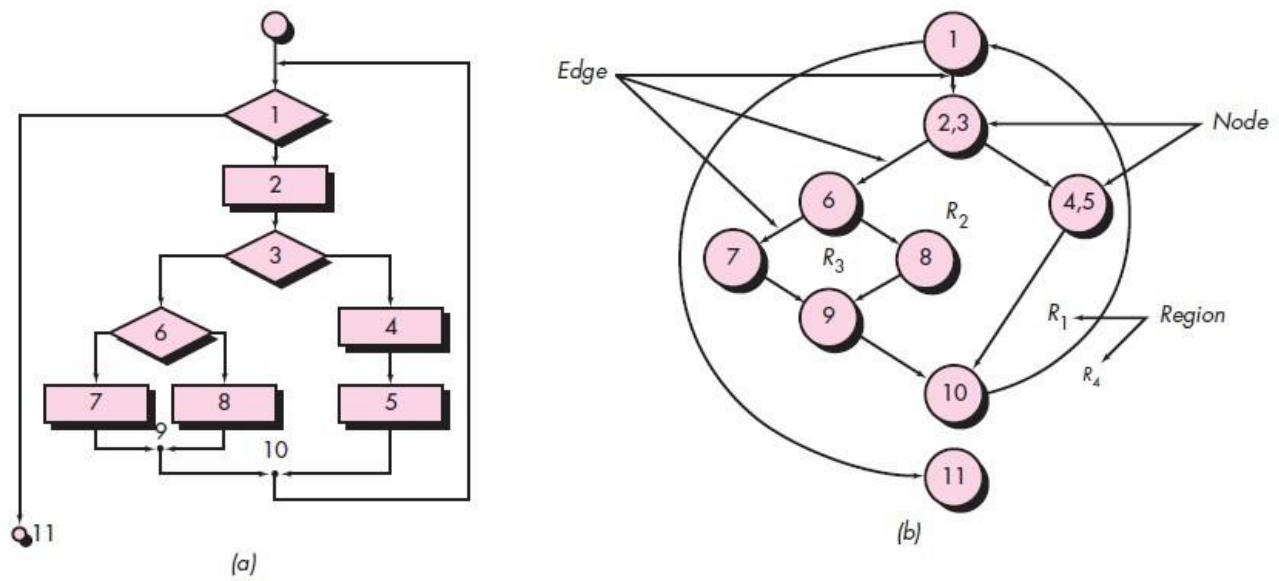


Fig b

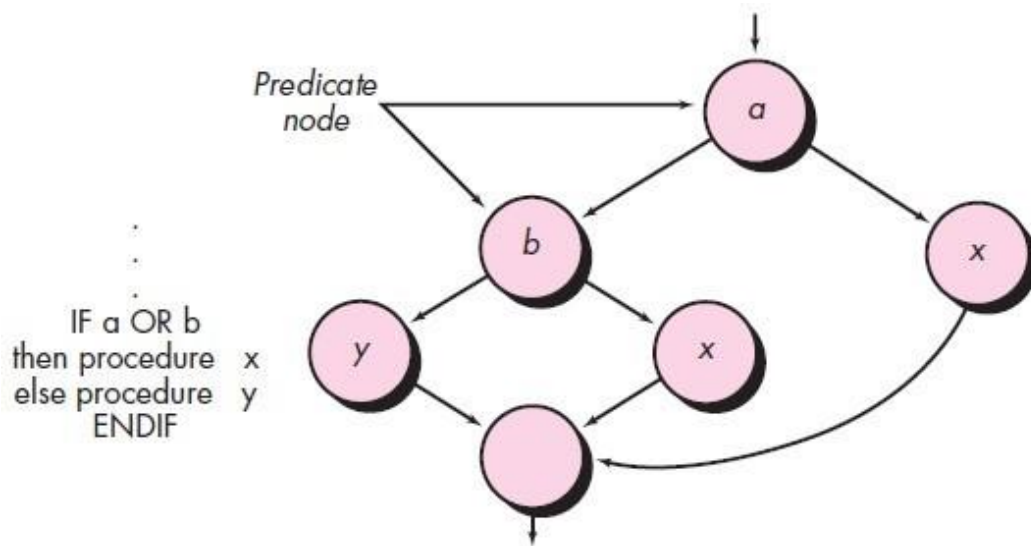


Fig c

When compound conditions are encountered in a procedural design, the generation of a flow graph becomes slightly more complicated. A compound condition occurs when one or more Boolean operators (logical OR, AND, NAND, NOR) is present in a conditional statement. Referring to Figure c, the program design language (PDL) segment translates into the flow graph shown. Note that a separate node is created for each of the conditions a and b in the statement IF a OR b. Each node that contains a condition is called a predicate node and is characterized by two or more edges emanating from it.

Independent Program Paths

An independent path is any path through the program that introduces at least one new set of processing statements or a new condition. When stated in terms of a flow graph, an independent path must move along at least one edge that has not been traversed before the path is defined. For example, a set of independent paths for the flow graph illustrated in Figure 18.2b is

Path 1: 1-11

Path 2: 1-2-3-4-5-10-1-11

Path 3: 1-2-3-6-8-9-10-1-11

Path 4: 1-2-3-6-7-9-10-1-11

Note that each new path introduces a new edge. The path

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

is not considered to be an independent path because it is simply a combination of already specified paths and does not traverse any new edges.

Paths 1 through 4 constitute a basis set for the flow graph in Figure 18.2b. That is, if you can design tests to force execution of these paths (a basis set), every statement in the program will have been guaranteed to be executed at least one time and every condition will have been executed on its true and false sides. It should be noted that the basis set is not unique. In fact, a number of different basis sets can be derived for a given procedural design.

How do you know how many paths to look for? The computation of cyclomatic complexity provides the answer. Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program. When used in the context of the basis path testing method, the value computed for cyclomatic complexity defines the number of independent paths in the basis set of a program and provides you with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once. Cyclomatic complexity has a foundation in graph theory and provides you with an extremely useful software metric.

Complexity is computed in one of three ways:

1. The number of regions of the flow graph corresponds to the cyclomatic complexity.

2. Cyclomatic complexity $V(G)$ for a flow graph G is defined as

$$V(G) = E - N + 2$$

where E is the number of flow graph edges and N is the number of flow graph nodes.

3. Cyclomatic complexity $V(G)$ for a flow graph G is also defined as

$$V(G) = P + 1$$

where P is the number of predicate nodes contained in the flow graph G .

Referring once more to the flow graph in Figure b, the cyclomatic complexity can be computed using each of the algorithms just noted:

The flow graph has four regions.

$$V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4.$$

$$V(G) = 3 \text{ predicate nodes} + 1 = 4.$$

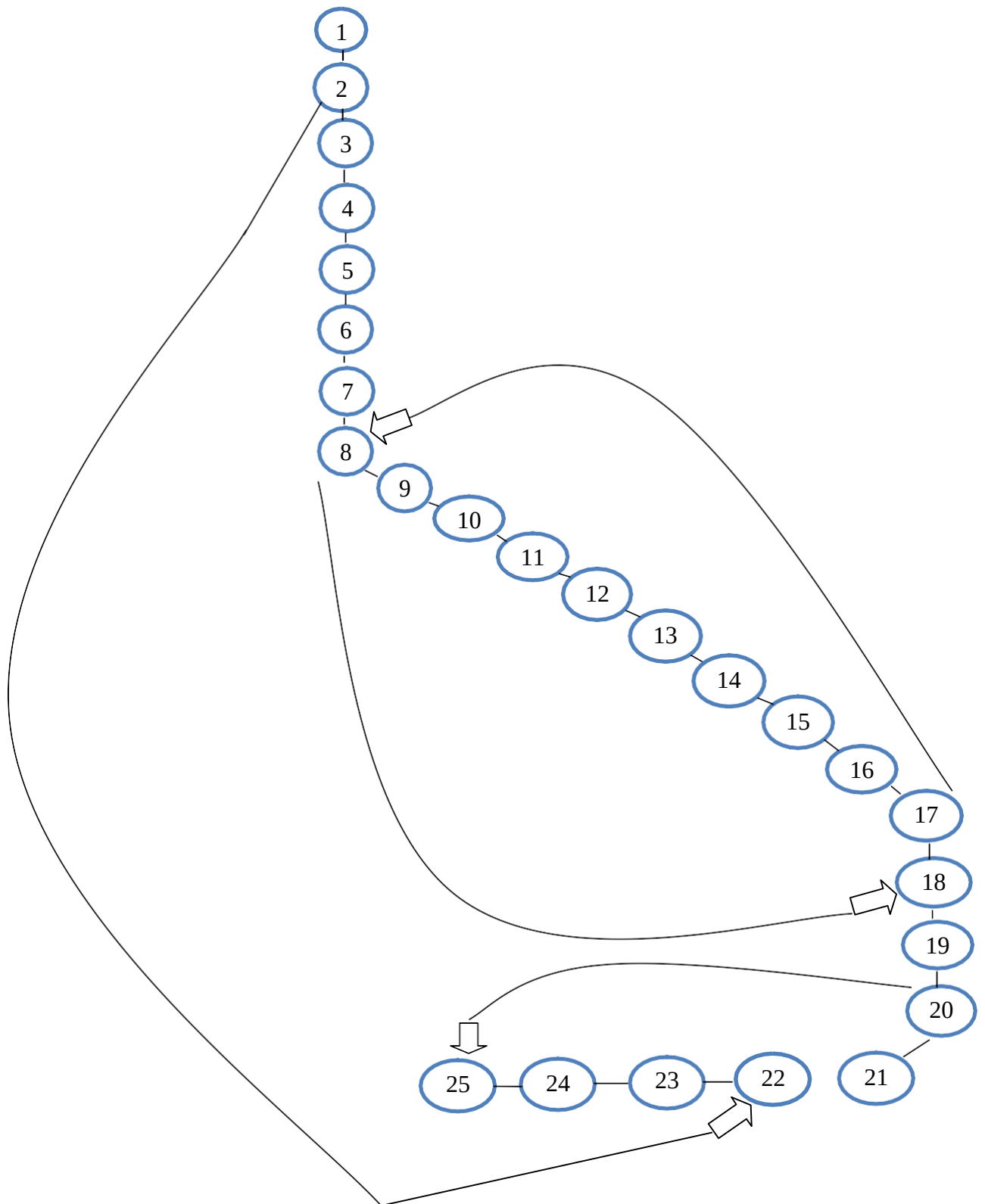
Therefore, the cyclomatic complexity of the flow graph in Figure b is 4.

More important, the value for $V(G)$ provides you with an upper bound for the number of independent paths that form the basis set and, by implication, an upper bound on the number of tests that must be designed and executed to guarantee coverage of all program statements.

BASIS PATH TESTING FOR DOCTOR VIEW PATIENT MODULE

```
1.    DefaultTabelmodel model = (DefaultTableModel)tablename.getModel1);
2.    try{
3.    class.forName("java.sql.Driver);
4.    Connection con= DriverManager.getConnection("jdbc:mysql://localhost/test","root","xyz");
5.    Statement stmt= con.createStatement();
6.    string query= "select * from Appointment where vid="+vid+f.getText+";"
7.    Result set rs= stmt.executeQuery(query);
8.    while(rs.next()){
9.    string uid=rs.getString("uid ");
10.   string name= rs.getString("Name");
11.   string address= rs.getString("Address");
12.   string mobile= rs.getString("Mobile");
13.   string Email= rs.getString("Email");
14.   string sex=rs.getString("Sex");
15.   string Age=rs.getString("Age");
16.   model.addRpw(newobject[ ]{uid,name,address,mobile,email,sex,age})
17.   }
18.   rs.close();
19.   stmt.close();
20.   con.close();
21.   }
22.   catch(Exception()) {
23..       JOptionPane.showMessageDialog(null,"Error in Connectivity");
24.   }
25.   }
```

1)PROGRAM FLOW GRAPH



2)CYCLOMATIC COMPLEXITY

CYCLOMATIC COMPLEXITY= $E-N+2$

E=NO. OF EDGES IN PROGRAM FLOW GRAPH

N=NO.OF NODES IN PROGRAM FLOW GRAPH

HERE E=27

N=25

SO CYCLOMATIC COMPLEXITY=4

Hence, there will be 4 independent paths.

3)INDEPENDENT PATHS

PATH A: 1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-16-17-18-19-20-25

PATH B: 1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-16-17-8-18-19-20-25

PATH C: 1-2-3-4-5-6-7-8-18-19-20-25

PATH D: 1-2-22-23-24-25

BLACKBOX TESTING

Black-box testing, also called behavioral testing, focuses on the functional requirements of the software. That is, black-box testing techniques enable you to derive sets of input conditions that will fully exercise all functional requirements for a program.

Black-box testing is not an alternative to white-box techniques. Rather, it is a complementary approach that is likely to uncover a different class of errors than whitebox methods.

Black-box testing attempts to find errors in the following categories: (1) incorrect or missing functions, (2) interface errors, (3) errors in data structures or external database access, (4) behavior or performance errors, and (5) initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, blackbox testing tends to be applied during later stages of testing . Because black-box testing purposely disregards control structure, attention is focused on the information domain. Tests are designed to answer the following questions:

- How is functional validity tested?
- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation?

By applying black-box techniques, you derive a set of test cases that satisfy the following criteria [Mye79]: (1) test cases that reduce, by a count that is greater than one, the number of additional test cases that must be designed to achieve reasonable testing, and (2) test cases that tell you something about the presence or absence of classes of errors, rather than an error associated only with the specific test at hand.

This testing has different variances like:

- 1) Boundary Value Analysis
- 2) Orthogonal Array Analysis
- 3) Graph-Based Testing methods
- 4) Equivalence Class Testing

EQUIVALENCE CLASS TESTING

Equivalence class testing is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived. An ideal test case single-handedly uncovers a class of errors (e.g., incorrect processing of all character data) that might otherwise require many test cases to be executed before the general error is observed.

Test-case design for equivalence partitioning is based on an evaluation of equivalence classes for an input condition. If a set of objects can be linked by relationships that are symmetric, transitive, and reflexive, an equivalence class is present [Bei95]. An equivalence class represents a set of valid or invalid states for input conditions. Typically, an input condition is either a specific numeric value, a range of values, a set of related values, or a Boolean condition. Equivalence classes may be defined according to the following guidelines:

1. If an input condition specifies a range, one valid and two invalid equivalence classes are defined.
2. If an input condition requires a specific value, one valid and two invalid equivalence classes are defined.
3. If an input condition specifies a member of a set, one valid and one invalid equivalence class are defined.

1) INPUT EQUIVALENCE CLASSES

VALID-I1={ pid}
INVALID-I2={pid<0}

2) OUTPUT EQUIVALENCE CLASSES

01={uid,name,email,sex,age,address,mobile}

02={Error in connectivity}

Chapter – 7

User Mannual

1.0 Introduction

The “HOSPITAL MANAGEMENT SYSTEM” application allows users a simple interface to access their account from a mobile device to view their details and appointments.

Like the doctors can check for their appointments and prescribe a given drug to a patient.

The patients can also book their appointment with Doctors This document will provide instructions for using the application .

2.0 Getting Started

Install the HOSPITAL MANAGEMENT SYSTEM” application . The application is compatible with

Windows 7 or above operating system

.

2.x Quick Start : USER

STEP 1 : Tap the “Login” icon in your device menu. The Sign up screen will be brought up.

STEP 2 : First time users need to sign up by providing their information (name , contact number and password) . “Register” will create their unique account .

STEP 3 : Recurring user already having account need to log in to use the application by providing their user name and password.

STEP 4 : Once you are logged in , you can perform specific functions like booking appointment or viewing appointment if you are a patient and prescribing drugs,viewing appointments if you are a doctor

STEP 5 : In case of query or complaint use “SUBMIT REQUEST” and “REGISTER COMPLAINT” options provided on the same page.

2. x System Requirements

Windows 7 or above operating system

Internet connection for first time login.

Troubleshooting

Missing or Incorrect Password or E-Mail

A message will be displayed in the event incorrect login information is entered. Try again with proper credentials to access

Chapter -8

Conclusion

The entire project has been developed and deployed as per the requirements stated by the user. It is found to be bug free as per the testing standards that are implemented.

The whole system's activities are divided into two major parts like User and admin. For implementing the system Android studio is used.

The system comprise of following features:

- Display bus number
- Calculation of fare
- Record requests from the user
- Record Complaints of the user

The estimated cost of the project is (efforts) 4 and the estimated size of the project is (FP) 153.

There are also few features which can be integrated with this system to make it more flexible. Below list shows the future points to be consider :

- Getting the current status of patient.
- Including a different module for pharmacy

Finally, we like to conclude that we put all our efforts throughout the development of our project and tried to fulfill most of the requirements of the user.

REFERENCES

1. IEEE Software Engineering Standards Committee, "IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications", October 20, 1998.

[IEEE] The applicable IEEE standards are published in "IEEE Standards Collection," 2001 edition.

2. The principle source of text book material is "Software Engineering A PRACTITIONER ' S APPROACH" by Roger S. Pressman.

3. The principle source of text book material is “A Concise Introduction to Software Engineering” by Pankaj Jalote.

